



Documentación del Plugin PosicionMaximosIntensidad.java para ImageJ/Fiji

Escuela de Física — Universidad de Costa Rica
Centro de Investigación en Ciencias Atómicas, Nucleares y Moleculares (CICANUM)

Jimena Hernández Tames^{*}
Documentación

Iván Rojas Duarte^{**}
Desarrollo del software

Supervisión del proyecto: **Erick Mora Ramírez**^{***}

24 de mayo de 2026

Descripción del software de análisis

1. Descripción general

El plugin `PosicionMaximosIntensidad.java` es un componente de software desarrollado para el entorno ImageJ/Fiji, diseñado con el propósito de extraer y registrar cuantitativamente la posición de los máximos de intensidad en *stacks* de imágenes de Tomografía por Emisión de Positrones (PET). Su creación se centra en el proyecto de control de calidad del escáner PET/CT Biograph Vision 450 Edge del CICANUM de la Universidad de Costa Rica (UCR), específicamente en la evaluación de la continuidad y estabilidad del movimiento de la camilla ante variaciones en la velocidad de adquisición.

La metodología experimental emplea cuatro fuentes radiactivas de ^{68}Ge posicionadas en los vértices de un cuadrado sobre la camilla del equipo, configurando un patrón geométrico de referencia conocido. Las imágenes DICOM resultantes son procesadas por este plugin para cuantificar el desplazamiento transversal y axial de las fuentes a lo largo de la dirección de avance de la camilla, permitiendo identificar irregularidades o perturbaciones en su movimiento.

^{*} maria.hernandeztames@ucr.ac.cr

^{**} ivan.rojasduarte@ucr.ac.cr

^{***} erick.mora@ucr.ac.cr

2. Entorno de ejecución y dependencias

El plugin se ejecuta sobre la plataforma ImageJ/Fiji (versión 2.x o superior) e implementa la interfaz `PlugIn` de la API estándar de ImageJ. No requiere la instalación de librerías externas ni dependencias adicionales más allá de las incluidas en la distribución base de Fiji.

Tabla 1: Especificaciones técnicas del entorno de ejecución.

Componente	Especificación
Plataforma	ImageJ / Fiji 2.x o superior
Lenguaje	Java (API estándar de ImageJ)
Dependencias externas	Ninguna
Formato de entrada	Stack DICOM (modalidad PET reconstruida)
Formato de salida	CSV y gráficas nativas de ImageJ

3. Descripción del código

El algoritmo se estructura en tres etapas secuenciales basadas en técnicas estándar de procesamiento digital de imágenes para el análisis de perfiles de intensidad espacial. La primera etapa establece un píxel de referencia global sobre el volumen completo, las siguientes etapas utilizan dicha referencia para generar perfiles de posición en las direcciones transversal y axial, respectivamente.

3.1. Etapa I: identificación del píxel de referencia global

El plugin recorre la totalidad de los *slices* del *stack* dentro de la Región de Interés (ROI) definida por el usuario. Este análisis estructural se basa en el procesamiento estandarizado de matrices tridimensionales en formato de secuencias de imágenes u objetos numéricos complejos de ImageJ [1]. Para cada píxel de coordenadas (x, y) en cada *slice* z , se evalúa su valor de intensidad. El píxel que presenta el valor máximo absoluto en todo el volumen define las coordenadas de referencia (x^*, y^*) , las cuales son reportadas en el registro de actividad (*Log*) de ImageJ. Esta referencia asume que alguna de las fuentes radiactivas exhibirá, en algún corte, la mayor intensidad del volumen completo.

3.2. Etapa II: perfil de desplazamiento transversal (plano Y^*)

Fijando la coordenada $y = y^*$, el algoritmo recorre, para cada *slice* z , la extensión horizontal de la ROI y localiza el valor de x que maximiza la intensidad en esa fila. El conjunto de tuplas $(z, x_{\text{máx}}, I_{\text{máx}})$ es exportado a un archivo CSV y graficado directamente en la interfaz de ImageJ, fundamentado en las técnicas de análisis y representación de perfiles de intensidad lineal descritas en la metodología oficial del software [1]. El perfil resultante, $x_{\text{máx}}$ en función de z , describe el desplazamiento transversal de las fuentes a lo largo de la dirección de avance de la camilla.

3.3. Etapa III: perfil de desplazamiento axial (plano X^*)

De forma análoga, fijando $x = x^*$, el algoritmo recorre verticalmente la ROI en cada *slice* z y localiza el valor de y que maximiza la intensidad en esa columna. El conjunto de tuplas $(z, y_{\text{máx}}, I_{\text{máx}})$ es exportado a un segundo archivo CSV y graficado en correspondencia con los principios de generación de trazas unidimensionales [1]. Este perfil describe el desplazamiento axial de las fuentes a lo largo del escaneo.

4. Archivos de salida

El plugin genera dos archivos en formato de valores separados por comas (CSV), cuyos nombres y contenido se detallan en la Tabla 2.

Tabla 2: Archivos CSV generados por el plugin.

Archivo	Columnas	Descripción
y_plane_profile.csv	z, x_max, intensity_max	Posición horizontal en la fila y^* por <i>slice</i>
x_plane_profile.csv	z, y_max, intensity_max	Posición vertical en la columna x^* por <i>slice</i>

Las coordenadas de posición se expresan en unidades de píxel relativas a la esquina superior izquierda de la ROI seleccionada, con indexación base cero (el primer *slice*, $z = 1$ en la interfaz de ImageJ, se registra como $z = 0$ en el CSV). La conversión a distancias físicas en milímetros requiere la aplicación del parámetro *PixelSpacing* consignado en el encabezado DICOM de las imágenes.

5. Consideraciones y limitaciones

(Se utiliza IA para analizar el código y dar recomendaciones, las cuales se estudian y se define cuáles podrían aplicarse para optimizar el código)

- **Estacionariedad del píxel de referencia:** Las coordenadas (x^*, y^*) se determinan una única vez al inicio del procesamiento y permanecen fijas durante el análisis. En escenarios donde el desplazamiento de la camilla sea apreciable, el plano de barrido podría alejarse de la posición real de las fuentes en *slices* distantes del máximo global, introduciendo un sesgo sistemático en la estimación de la posición.
- **Sensibilidad a la definición de la ROI:** Si alguna fuente excede los límites de la ROI en algún *slice*, el máximo detectado en ese corte corresponderá a fondo u otra estructura, comprometiendo la validez del dato. Se recomienda definir una ROI con margen suficiente para contener el desplazamiento esperado de las fuentes en toda la extensión axial del escaneo.
- **Resolución espacial discreta:** La posición reportada corresponde al centroide del píxel de máxima intensidad. En ausencia de ajuste sub-píxel, la resolución efectiva de posición está limitada por el tamaño del píxel de la imagen reconstruida.
- **Unidades de salida:** Todos los valores de posición se expresan en píxeles. Su interpretación física requiere la multiplicación por el factor *PixelSpacing* del encabezado DICOM.

6. Instrucciones de uso y guía

A continuación, se detalla el procedimiento secuencial para la instalación, configuración y ejecución del plugin en el software ImageJ/Fiji. Este manual está diseñado para asegurar la reproducibilidad del análisis por parte de cualquier operador científico o técnico del laboratorio.

Fase 1: Instalación y compilación del Plugin

Debido a que el algoritmo se suministra en código fuente (`.java`), ImageJ requiere compilarlo de forma interna para generar el archivo ejecutable de la biblioteca (`.class`).

1. Asegúrese de que el archivo del código fuente esté guardado estrictamente con el nombre `PosicionMaximosIntensidad.java`. Si el nombre del archivo difiere de la clase pública declarada en el código, el compilador de Java generará un error de sintaxis de forma inmediata.

2. Instalación mediante la interfaz:

- Abra el software ImageJ/Fiji.
- En la barra de menú superior, diríjase a **Plugins** → **Install..**
- Se desplegará un explorador de archivos. Busque y seleccione el archivo `PosicionMaximosIntensidad.java` que guardó previamente y haga clic en *Abrir* (*Open*).

Note: Si la instalación no funciona de esta manera, copie el archivo `PosicionMaximosIntensidad.java` e infíltrelo dentro de la carpeta interna llamada **plugins** en el directorio raíz de su instalación de ImageJ/Fiji.

3. El software compilará el código de forma automática. Al finalizar, el nuevo comando quedará registrado y visible en la parte inferior del menú desplegable de **Plugins**.

Fase 2: Preparación e importación de las imágenes PET/CT

1. **Carga del Stack de imágenes:** Inicie la importación de la secuencia de cortes tomográficos DICOM dirigiéndose al menú superior y seleccionando la ruta:

`File` → `Import` → `Image Sequence...`

2. En la ventana emergente, seleccione la carpeta que contiene exclusivamente los archivos DICOM del estudio PET/CT. Haga clic sobre la **primera imagen** de la secuencia y presione *Open*.
3. Aparecerá una ventana de diálogo flotante (*Sequence Options*). Deje las opciones predeterminadas de ordenamiento por nombre y presione *OK*.

Fase 3: Delimitación de la Región de Interés (ROI)

La correcta delimitación del área de cálculo previene que el algoritmo capture ruidos espaciales o artefactos fuera del área del barrido físico de la camilla.

1. En la barra de herramientas principal de ImageJ, seleccione el ícono del rectángulo (*Rectangle Tool*).
2. Dibuje una Región de Interés (ROI) rectangular sobre la imagen que encierre de manera holgada a las fuentes radiactivas.

Fase 4: Funcionamiento del plugin para extraer los datos relevantes

Debido al error que se presentó al momento de llegar a esta fase (se cree un problema de instalación del plugin), no se lograron realizar estos pasos, pero según lo mostrado por Iván Rojas, se sigue el siguiente proceso

1. **Llamar script:** Con la imagen activa y la ROI dibujada en pantalla, diríjase al menú superior y haga clic en:

`Plugins` → `PosicionMaximosIntensidad`

2. **Fase I automática:** El programa ejecutará la búsqueda del píxel más denso de todo el volumen en segundo plano. Al terminar esta búsqueda, la consola de registro (*Log Window*) de ImageJ se abrirá automáticamente en la pantalla mostrando las coordenadas calculadas del máximo de referencia: **Plano Y* = [valor]** y **Plano X* = [valor]**.
3. **Exportación del perfil transversal (Fase II):** Inmediatamente después, surgirá una ventana del sistema "*Guardar perfil Y**". Seleccione la carpeta de destino en su computadora, asigne el nombre deseado al archivo (por defecto se sugiere `y_plane_profile.csv`) y presione *Guardar*.
4. **Exportación del perfil axial (Fase III):** Seguidamente, aparecerá una segunda ventana "*Guardar perfil X**". Repita el proceso seleccionando su directorio de trabajo, asigne el nombre (por defecto `x_plane_profile.csv`) y presione *Guardar*.

Fase 5: Visualización e interpretación de resultados

Una vez completadas las fases de almacenamiento, se espera que el plugin despliegue de forma automática los resultados gráficos sobre la interfaz, según lo observado en la reunión

1. Se abrirán de forma automática dos ventanas gráficas nativas de ImageJ tituladas "*Perfil Y**" y "*Perfil X**". Estas curvas muestran el número de corte o *slice* (z) en el eje de las abscisas y la posición del píxel de máxima intensidad en las ordenadas.
2. **Resultados esperados, según pruebas anteriores**
 - Una línea horizontal perfectamente recta y plana en estas gráficas refleja que la camilla se mueve de forma ideal y lineal, manteniendo las fuentes en una posición transversal fija y constante respecto al detector.
 - Cualquier salto abrupto, oscilación o escalón discontinuo en la gráfica representa una desviación milimétrica en el desplazamiento continuo de la camilla a esa velocidad de escaneo específica.

A. Anexo: Código Fuente del Plugin

A continuación se adjunta el código fuente completo del archivo `PosicionMaximosIntensidad.java` desarrollado por Iván Rojas Duarte para el entorno ImageJ/Fiji. Este algoritmo implementa de forma explícita las tres etapas de análisis descritas en esta documentación para evaluar la estabilidad en el desplazamiento de la camilla del PET/CT.

```
1 import ij.*;
2 import ij.plugin.*;
3 import ij.process.*;
4 import ij.gui.*;
5 import ij.io.SaveDialog;
6 import java.io.*;
7 import java.awt.Rectangle;
8 import ij.gui.Plot;
9
10 // Define el plugin implementando la interfaz PlugIn
11 public class PosicionMaximosIntensidad implements PlugIn {
12
13     // Metodo principal que ejecuta el plugin
14     public void run(String arg) {
15
16         // Obtiene la imagen activa en ImageJ
17         ImagePlus imp = IJ.getImage();
18         // Obtiene la ROI (region de interes) seleccionada
19         Roi roi = imp.getRoi();
20
21         // Verifica si no hay ROI seleccionada
22         if (roi == null) {
23             // Muestra un mensaje de error si no hay ROI
24             IJ.error("Debe seleccionar una ROI.");
25             return;
26         }
27
28         // Obtiene el rectangulo delimitador de la ROI
29         Rectangle bounds = roi.getBounds();
30
31         // Ancho de la ROI en pixeles
32         int width = bounds.width;
33         // Alto de la ROI en pixeles
34         int height = bounds.height;
35         // Numero total de planos (slices) en el stack
36         int slices = imp.getStackSize();
37
38         // =====
39         // 1. Planos de interes
40         // =====
41
42         // Inicializa el maximo global con el menor valor posible
43         double globalMax = -Double.MAX_VALUE;
44         // Coordenada X del pixel con maxima intensidad global
45         int xStar = -1;
46         // Coordenada Y del pixel con maxima intensidad global
47         int yStar = -1;
48
49         // Itera sobre cada slice del stack
50         for (int z = 1; z <= slices; z++) {
51             // Activa el slice z en la imagen
```

```

52         imp.setSlice(z);
53         // Obtiene el procesador de imagen del slice actual
54         ImageProcessor ip = imp.getProcessor();
55
56         // Itera sobre cada fila de la ROI
57         for (int y = 0; y < height; y++) {
58             // Itera sobre cada columna de la ROI
59             for (int x = 0; x < width; x++) {
60
61                 // Obtiene el valor del pixel en coordenadas
62                 absolutas double val = ip.getPixelValue(bounds.x + x, bounds.y
63                 + y);
64
65                 // Comprueba si el valor actual supera el maximo
66                 global
67                 if (val > globalMax) {
68                     // Actualiza el maximo global
69                     globalMax = val;
70                     // Guarda la coordenada X del nuevo maximo
71                     xStar = x;
72                     // Guarda la coordenada Y del nuevo maximo
73                     yStar = y;
74                 }
75             }
76
77             // Imprime en el log la fila del maximo global encontrado
78             IJ.log("Plano Y* = " + yStar);
79             // Imprime en el log la columna del maximo global encontrado
80             IJ.log("Plano X* = " + xStar);
81
82             // =====
83             // Maximos de intensidad en X para cada Slice
84             // =====
85
86             // Abre dialogo para guardar el perfil del plano Y*
87             SaveDialog sdY = new SaveDialog("Guardar perfil Y*", "
y_plane_profile", ".csv");
88             // Verifica que el usuario haya elegido un nombre de archivo
89             if (sdY.getFileName() != null) {
90
91                 try {
92                     // Crea el archivo CSV en la ruta seleccionada
93                     BufferedWriter writer = new BufferedWriter(
94                         new FileWriter(sdY.getDirectory() + sdY.getFileName
95                         ())
96                     );
97
98                     // Escribe la cabecera del CSV
99                     writer.write("z,x_max,intensity_max\n");
100                     // Array para almacenar los indices de slice (eje Z)
101                     double[] zVals = new double[slices];
102                     // Array para almacenar las posiciones X del maximo por
103                     slice
104                     double[] xVals = new double[slices];
105
106                     // Itera sobre cada slice del stack

```

```

105         for (int z = 1; z <= slices; z++) {
106
107             // Activa el slice z
108             imp.setSlice(z);
109             // Obtiene el procesador del slice actual
110             ImageProcessor ip = imp.getProcessor();
111
112             // Inicializa el maximo local con el menor valor
113             posible double maxVal = -Double.MAX_VALUE;
114             // Inicializa la posicion X del maximo local
115             int xMax = -1;
116
117             // Recorre todos los pixeles en la fila Y* de la ROI
118             for (int x = 0; x < width; x++) {
119
120                 // Obtiene el valor del pixel en la fila Y* del
121                 slice actual double val = ip.getPixelValue(bounds.x + x,
122                 bounds.y + yStar);
123
124                 // Comprueba si supera el maximo local
125                 if (val > maxVal) {
126                     // Actualiza el maximo local
127                     maxVal = val;
128                     // Guarda la posicion X del nuevo maximo
129                     local xMax = x;
130                 }
131             }
132             // Almacena el indice del slice (base 0) para la
133             grafica zVals[z-1] = z-1;
134             // Almacena la posicion X del maximo para la grafica
135             xVals[z-1] = xMax;
136             // Escribe la fila de datos en el CSV
137             writer.write((z-1) + "," + xMax + "," + maxVal + "\n");
138
139         }
140
141         // Cierra el archivo CSV
142         writer.close();
143         // Crea la grafica de posicion X vs slice en el plano Y*
144         Plot plotY = new Plot("Perfil Y*", "Slice (z)", "X(pixel
145         )", zVals, xVals);
146         // Muestra la grafica en pantalla
147         plotY.show();
148
149     } catch (IOException e) {
150         // Muestra un error si falla la escritura del archivo
151         IJ.error(e.getMessage());
152     }
153
154     // =====
155     // Posicion de maximos de intensidad en Y para cada Z (se
156     // desarrollan los mismos pasos que para el plano Y*, pero ahora para el
157     // plano X*)
158     // =====

```



```

155
156
157     SaveDialog sdX = new SaveDialog("Guardar perfil X*", "
x_plane_profile", ".csv");
158
159     if (sdX.getFileName() != null) {
160         try {
161             BufferedWriter writer = new BufferedWriter(
162                 new FileWriter(sdX.getDirectory() + sdX.getFileName
163                 ())
164                 );
165
166             writer.write("z,y_max,intensity_max\n");
167
168             double[] yVals = new double[slices];
169             double[] zVals = new double[slices];
170
171
172             for (int z = 1; z <= slices; z++) {
173
174                 imp.setSlice(z);
175                 ImageProcessor ip = imp.getProcessor();
176
177                 double maxVal = -Double.MAX_VALUE;
178                 int yMax = -1;
179
180                 for (int y = 0; y < height; y++) {
181
182                     double val = ip.getPixelValue(bounds.x + xStar,
183                     bounds.y + y);
184
185                     if (val > maxVal) {
186                         maxVal = val;
187                         yMax = y;
188                     }
189
190                     zVals[z-1] = z-1;
191                     yVals[z-1] = yMax;
192
193                     writer.write((z-1) + "," + yMax + "," + maxVal + "\n
194                     ");
195                 }
196
197                 writer.close();
198                 Plot plotX = new Plot("Perfil X*", "Slice (z)", "Y(pixel
199                 )", zVals, yVals);
200                 plotX.show();
201             } catch (IOException e) {
202                 IJ.error(e.getMessage());
203             }
204
205             // Muestra mensaje de confirmacion al finalizar el plugin
206             IJ.showMessage("Procesamiento completo.");
207         }
208     }

```

Listing 1: Código completo del plugin `PosicionMaximosIntensidad.java`.

Referencias

- [1] T. Ferreira and W. Rasband. *ImageJ User Guide: Image Stacks and Profile Plots (Version 1.46r)*. National Institutes of Health.